DATA 622 Lab 5: Ames Housing Data with Trees

Sabina Baraili

Problem 1: Building up to a Baseline Model

(a) Basic EDA and Feature Selection

The response variable is `SalePrice`, and I modeled `logSalePrice` instead of raw sale price because the original distribution is right-skewed. I also created binary indicators for features like pools and fireplaces, recoded quality variables as ordinal scores, filled selected missing categories, and collapsed rare levels into `"Others"`.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn.model_selection as skm
import sklearn.linear_model as skl
import xgboost as xgb

from scipy import stats
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error
from sklearn.tree import DecisionTreeRegressor as DTR
from sklearn.ensemble import RandomForestRegressor as RF
from sklearn.tree import plot_tree
```

```
df = pd.read_csv(
    "https://raw.githubusercontent.com/"
    "georgehagstrom/DATA622Spring2026/main/"
    "website/assignments/labs/labData/AmesHousing.csv"
)

df.columns = df.columns.str.replace(" ", "", regex=False)
```

```
df["logSalePrice"] = np.log(df["SalePrice"])
df["hasPool"] = df["PoolQC"].notna().astype(int)
df["hasFireplace"] = (df["Fireplaces"] >= 1).astype(int)

qual_map = {"Ex": 4, "Gd": 3, "TA": 2, "Fa": 1, "Po": 0}
bsmt_map = {"Ex": 5, "Gd": 4, "TA": 3, "Fa": 2, "Po": 1, "NA": 0}

df["KitchenQual"] = df["KitchenQual"].map(qual_map)
df["ExterQual"] = df["ExterQual"].map(qual_map)
df["BsmtQual"] = df["BsmtQual"].fillna("NA").map(bsmt_map)
df["Alley"] = df["Alley"].fillna("NoAlley")
df["MiscFeature"] = df["MiscFeature"].fillna("NoMiscFeature")
df["GrLivArea"] = df["GrLivArea"].astype(float)
```

```
ordinal_cols = [
    "ExterQual", "BsmtQual", "KitchenQual",
    "TotRmsAbvGrd", "FullBath", "OverallQual"
]

categorical_cols = [
    "Neighborhood", "MSZoning", "BldgType", "HouseStyle",
    "CentralAir", "Foundation", "Street", "hasPool",
    "hasFireplace", "MSSubClass", "LotShape", "LandContour",
    "Utilities", "MiscFeature", "LotConfig", "LandSlope",
    "Condition1", "RoofStyle", "Heating", "SaleType",
    "SaleCondition"
]

numeric_cols = [
    "GrLivArea", "LotArea", "1stFlrSF",
    "YearBuilt", "OpenPorchSF", "YrSold"
]

for col in categorical_cols:
    counts = df[col].value_counts()
    rare_levels = counts[counts < 30].index
    df[col] = df[col].replace(rare_levels, "Others").astype(str)

X = df[ordinal_cols + categorical_cols + numeric_cols].copy()
y = df["logSalePrice"].copy()

X_train, X_test, y_train, y_test = skm.train_test_split(
    X, y, test_size=0.3, random_state=2026
)
```

For the numeric and ordinal variables, I used the training-set correlation structure to remove variables with little relationship to `logSalePrice`. The heatmap shows that `LotArea` and

`YrSold` had weak correlation with the response.

```
correlation_matrix = pd.concat(
    [y_train, X_train[ordinal_cols + numeric_cols]], axis=1
).corr()

plt.figure(figsize=(13, 15))
sns.heatmap(
    correlation_matrix,
    annot=True,
    fmt=".3f",
    cmap="RdBu",
    center=0,
    linewidths=0.5,
    square=True,
    cbar_kws={"label": "Correlation Coefficient"},
    annot_kws={"size": 7}
)
plt.title("Correlation Heatmap for logSalePrice and Numeric Features")
plt.xticks(rotation=45, ha="right")
plt.yticks(rotation=0)
plt.show()
```

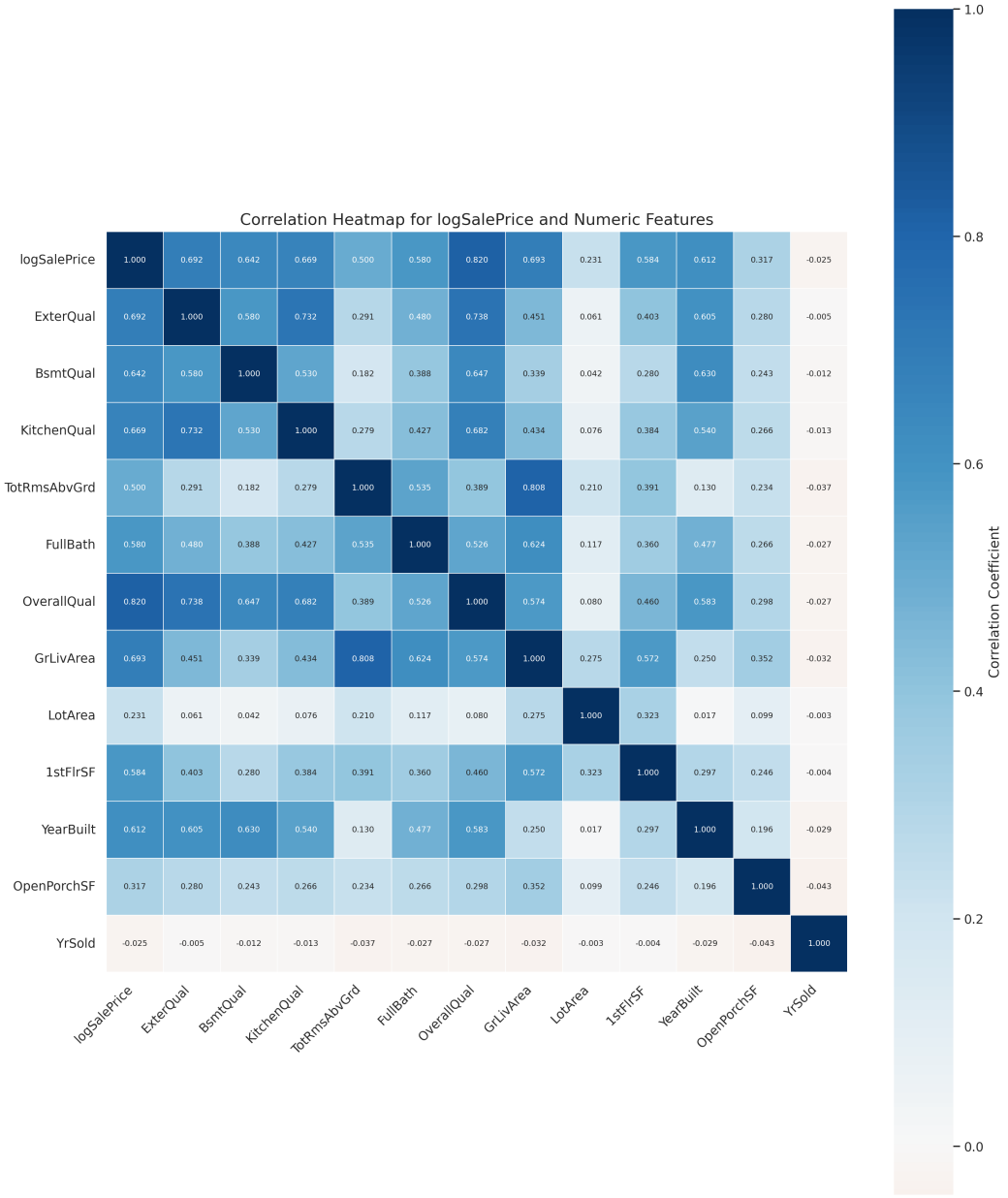

Figure 1: Correlation heatmap for `logSalePrice` and numeric/ordinal predictors.

For the categorical predictors, I used one-way ANOVA and kept the features with significant mean differences in `logSalePrice`.

```
significant_features = []
yX_train_cat = pd.concat([y_train, X_train[categorical_cols]], axis=1)
```

```
for col in categorical_cols:
    groups = [
        group["logSalePrice"].values
        for _, group in yX_train_cat.groupby(col)
    ]
    _, p_val = stats.f_oneway(*groups)
    if p_val < 0.01:
        significant_features.append(col)

categorical_features = significant_features
numeric_features = ["GrLivArea", "1stFlrSF", "YearBuilt", "OpenPorchSF"]
ordinal_features = ordinal_cols.copy()
```

This left 28 final predictors: 6 ordinal, 18 categorical, and 4 numeric variables.

Interpretation

In simple terms, this section cleaned the data and kept the housing characteristics that are most likely to explain price, especially size, quality, age, and neighborhood.

(b) Baseline Ridge Regression Model

I used a preprocessing pipeline that standardizes the ordinal and numeric predictors and one-hot encodes the categorical variables. Then I tuned ridge regression by cross-validation.

```
preprocessor = ColumnTransformer(
    transformers=[
        ("ord_num", StandardScaler(), ordinal_features + numeric_features),
        (
            "cat",
            OneHotEncoder(
                drop="first",
                sparse_output=False,
                handle_unknown="ignore"
            ),
            categorical_features,
        ),
    ]
)

alphas = np.logspace(6, -2, 100)
kfold = skm.KFold(10, random_state=2026, shuffle=True)
ridge_cv = skl.RidgeCV(
    alphas=alphas,
```

```
    cv=kfold,
    scoring="neg_mean_squared_error"
)

ridge_pipeline = Pipeline(
    steps=[("preprocess", preprocessor), ("ridge", ridge_cv)]
)

ridge_pipeline.fit(X_train, y_train)
ridge_pred = ridge_pipeline.predict(X_test)
ridge_test_mse = mean_squared_error(y_test, ridge_pred)
```

The tuned ridge model chose `alpha = 8.1113` and achieved a test MSE of `0.0198`.

```
feature_names = ridge_pipeline.named_steps["preprocess"].get_feature_names_out()
feature_importance = pd.DataFrame(
    {
        "Feature": feature_names,
        "Coefficient": ridge_pipeline.named_steps["ridge"].coef_,
    }
)

feature_importance["Abs_Coefficient"] = (
    feature_importance["Coefficient"].abs()
)

top20 = (
    feature_importance
    .sort_values("Abs_Coefficient", ascending=False)
    .head(20)
)

plt.figure(figsize=(10, 7))
sns.barplot(data=top20, x="Coefficient", y="Feature")
plt.axvline(0, color="black", linewidth=1)
plt.title("Top 20 Ridge Coefficients")
plt.show()
```

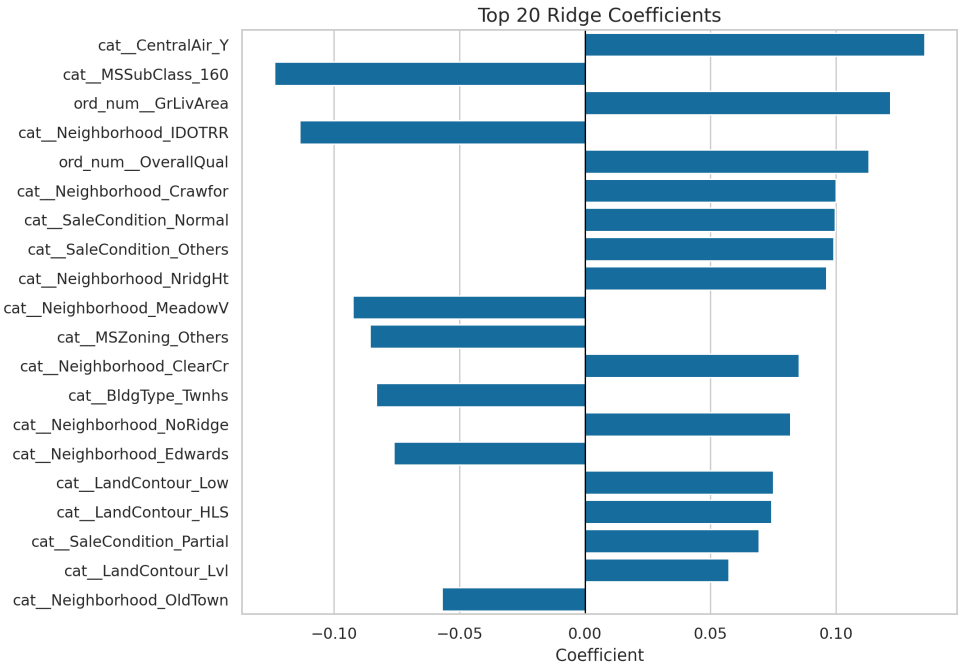

Figure 2: Top 20 ridge coefficients from the baseline model.

The largest positive effects are associated with better overall quality, larger above-ground living area, and central air.

Interpretation

The ridge model gives a strong baseline. Larger homes and higher-quality homes generally sell for more, while some neighborhoods and dwelling types are linked to lower prices.

Problem 2: Decision Trees and Random Forests

(a) Simple Decision Tree

To keep the tree readable, I fit a regression tree with `max_depth = 5`.

```
tree_preprocessor = ColumnTransformer(
    transformers=[
        (
            "cat",
            OneHotEncoder(
                drop="first",
                sparse_output=False,
```

```
                handle_unknown="ignore"
            ),
            categorical_features,
        ),
        ("ord_num", "passthrough", ordinal_features + numeric_features),
    ]
)

tree_pipeline = Pipeline(
    [
        ("preprocess", tree_preprocessor),
        ("tree", DTR(max_depth=5, random_state=2026)),
    ]
)

tree_pipeline.fit(X_train, y_train)
tree_pred = tree_pipeline.predict(X_test)
tree_mse = mean_squared_error(y_test, tree_pred)
```

The resulting test MSE was `0.0374`.

```
plt.figure(figsize=(28, 16))
plot_tree(
    tree_pipeline.named_steps["tree"],
    feature_names=tree_pipeline.named_steps["preprocess"].get_feature_names_out(),
    filled=True,
    rounded=True,
    fontsize=9,
    precision=2,
)
plt.title("Decision Tree Regression (max_depth = 5)")
plt.show()
```

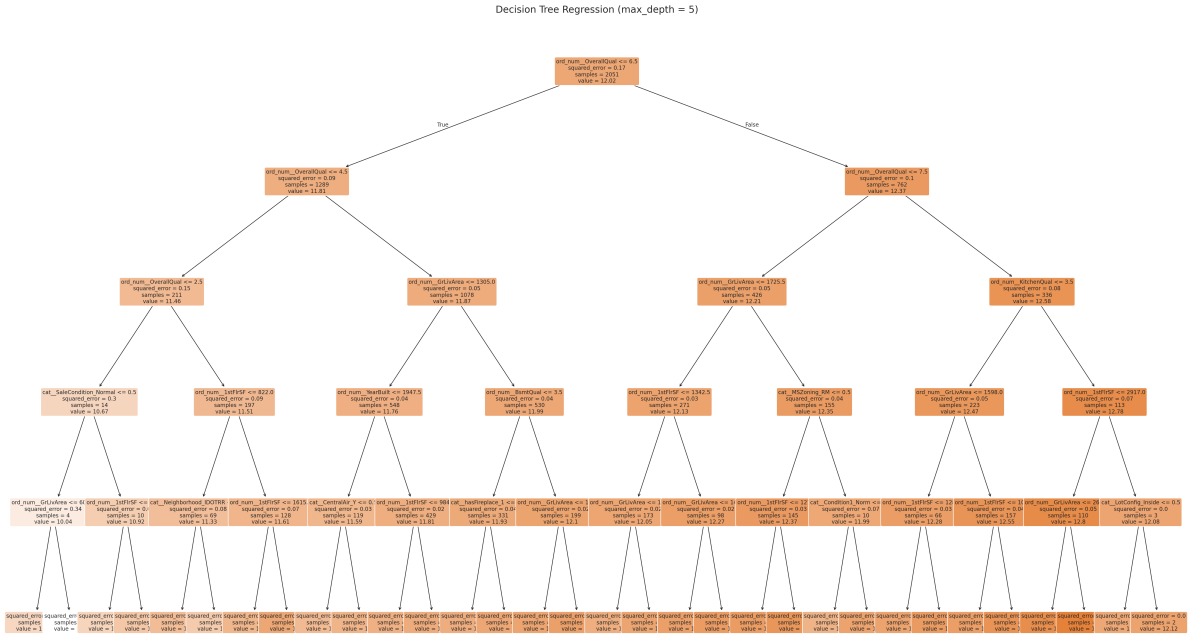

Figure 3: Decision tree regression with depth limited to 5.

The first split is on `OverallQual`, showing that overall house quality is the most important single rule in the tree.

Interpretation

The simple tree is easy to explain because it uses yes-or-no rules. The first and strongest split shows that quality is the main driver of house prices in this model.

(b) Tree Pruning

I then used cost-complexity pruning and tuned the pruning parameter by cross-validation.

```
X_train_transformed = tree_preprocessor.fit_transform(X_train)
raw_tree = DTR(random_state=2026)
ccp_path = raw_tree.cost_complexity_pruning_path(
    X_train_transformed, y_train
)

pruned_tree_pipeline = Pipeline(
    [
        ("preprocess", tree_preprocessor),
```

```
        ("tree", DTR(max_depth=15, random_state=2026)),
    ]
)

pruned_grid = skm.GridSearchCV(
    pruned_tree_pipeline,
    param_grid={"tree__ccp_alpha": ccp_path.ccp_alphas},
    cv=skm.KFold(n_splits=5, shuffle=True, random_state=2026),
    scoring="neg_mean_squared_error",
    refit=True,
)

pruned_grid.fit(X_train, y_train)
pruned_pred = pruned_grid.predict(X_test)
pruned_test_mse = mean_squared_error(y_test, pruned_pred)
```

The best pruning value was approximately `0.0001395`, and the pruned tree improved the test MSE to `0.0327`.

```
cv_mses = -pruned_grid.cv_results_["mean_test_score"]
alphas = ccp_path.ccp_alphas
mask = alphas > 0

plt.figure(figsize=(10, 6))
plt.plot(alphas[mask], cv_mses[mask], "-o", markersize=3)
plt.axvline(
    pruned_grid.best_params_["tree__ccp_alpha"],
    color="red",
    linestyle="--"
)
plt.xscale("log")
plt.xlabel("Cost Complexity Alpha")
plt.ylabel("Cross-Validated MSE")
plt.title("Pruned Tree Cross-Validation Curve")
plt.show()
```

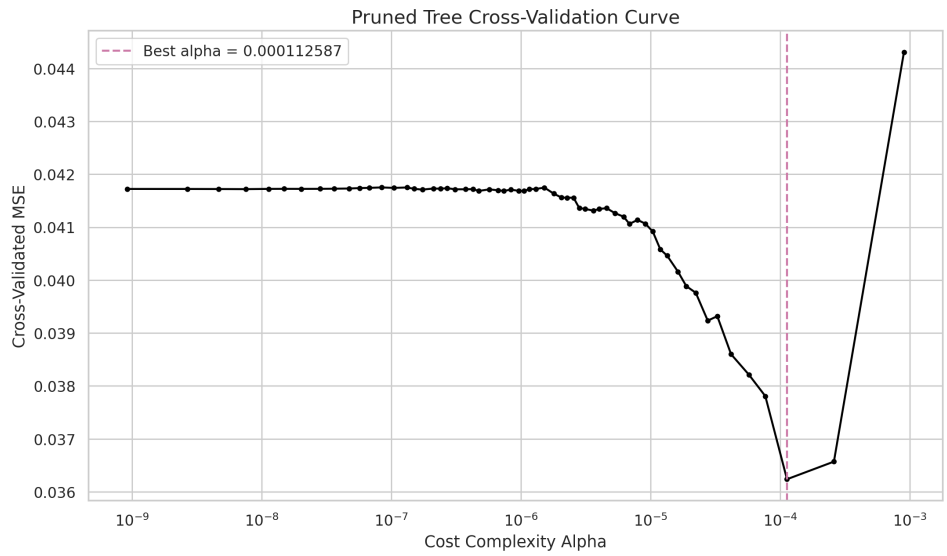

Figure 4: Cross-validation curve for cost-complexity pruning.

Interpretation

Pruning improves the balance between fit and generalization. It keeps useful structure while removing branches that do not help enough on new data.

(c) Random Forests

For the random forest, I tuned `max_features` by cross-validation.

```
rf_pipeline = Pipeline(
    [
        ("preprocess", tree_preprocessor),
        ("rf", RF(random_state=2026, n_jobs=-1)),
    ]
)

rf_grid = skm.GridSearchCV(
    rf_pipeline,
    param_grid={"rf__max_features": list(range(4, 81))},
    cv=skm.KFold(n_splits=5, shuffle=True, random_state=2026),
    scoring="neg_mean_squared_error",
)

rf_grid.fit(X_train, y_train)
rf_pred = rf_grid.predict(X_test)
rf_test_mse = mean_squared_error(y_test, rf_pred)
```

The final random forest test MSE was `0.0197`, which was the best performance among the main models.

```
plt.figure(figsize=(10, 6))
plt.plot(
    list(range(4, 81)),
    -rf_grid.cv_results_["mean_test_score"]
)
plt.xlabel("max_features")
plt.ylabel("Validation MSE")
plt.title("Random Forest Tuning Curve")
plt.show()
```

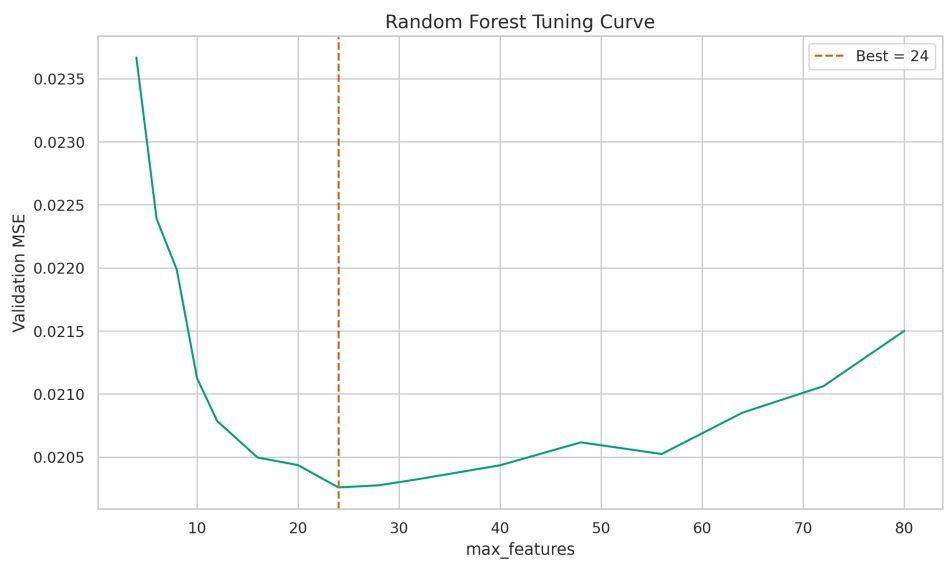

Figure 5: Random forest validation MSE as `max_features` changes.

```
best_rf = rf_grid.best_estimator_.named_steps["rf"]
rf_feature_names = (
    rf_grid.best_estimator_.named_steps["preprocess"].get_feature_names_out()
)

rf_importance_df = pd.DataFrame(
    {
        "Feature": rf_feature_names,
        "Importance": best_rf.feature_importances_,
    }
).sort_values("Importance", ascending=False)

plt.figure(figsize=(10, 6))
```

```
sns.barplot(data=rf_importance_df.head(15), x="Importance", y="Feature")
plt.title("Top Random Forest Feature Importances")
plt.show()
```

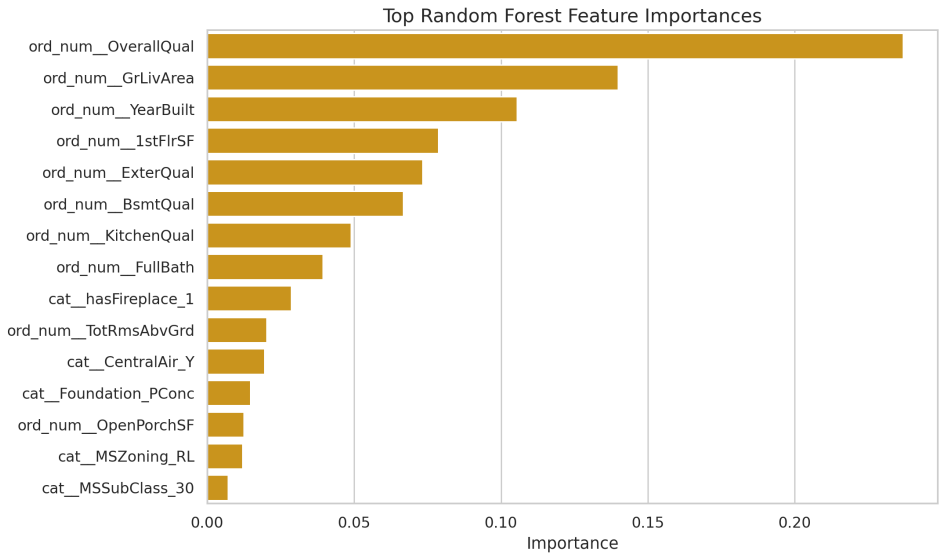

Figure 6: Top random forest feature importances.

The most important features were `OverallQual`, `GrLivArea`, and `ExterQual`.

Interpretation

The random forest performs well because it combines many trees and captures more complicated patterns than a single tree while still emphasizing the same housing characteristics.

(d) Comparing Ridge Regression and Random Forests

To compare the behavior of the two models, I plotted partial dependence for `GrLivArea`.

```
grliv_idx = X_train.columns.to_list().index("GrLivArea")
grid_values = np.linspace(0, 15000, 100)

ridge_pdp = []
rf_pdp = []

for val in grid_values:
    X_temp = X_train.copy()
```

```
    X_temp.iloc[:, grliv_idx] = val
    ridge_pdp.append(ridge_pipeline.predict(X_temp).mean())
    rf_pdp.append(rf_grid.predict(X_temp).mean())

fig, axes = plt.subplots(1, 2, figsize=(12, 5), sharey=True)
axes[0].plot(grid_values, ridge_pdp)
axes[0].set_title("Ridge Regression")
axes[1].plot(grid_values, rf_pdp)
axes[1].set_title("Random Forest")
plt.show()
```

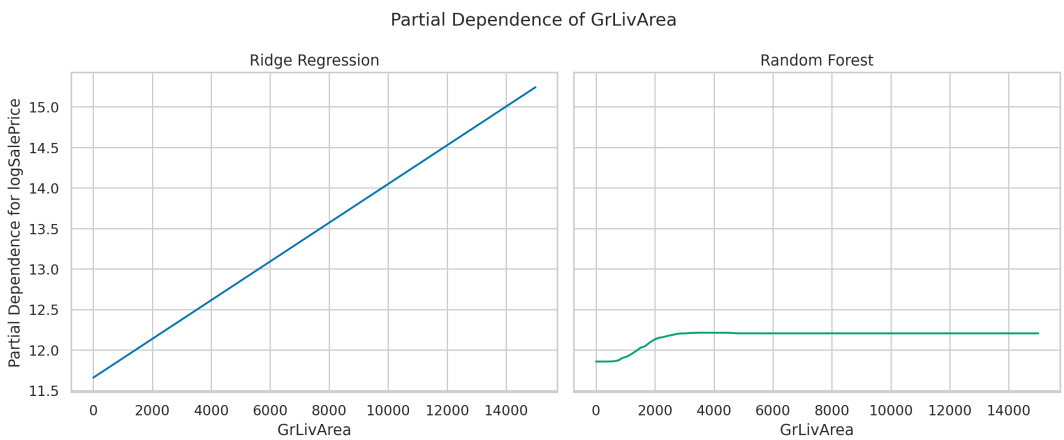

Figure 7: Partial dependence comparison for `GrLivArea` under ridge regression and random forest.

The ridge model increases smoothly, while the random forest flattens once the predictor moves outside the training range.

Interpretation

Ridge regression extrapolates smoothly, while the random forest is more flexible inside the observed range but tends to level off outside the data it has seen.

Problem 3: Boosting

(a) Default XGBoost

I fit a default `XGBRegressor` on the same processed feature matrix used for the tree-based models.

```
X_train_transformed = tree_preprocessor.fit_transform(X_train)
X_test_transformed = tree_preprocessor.transform(X_test)

xgb_default = xgb.XGBRegressor(random_state=2026)
xgb_default.fit(X_train_transformed, y_train)
xgb_default_pred = xgb_default.predict(X_test_transformed)
xgb_default_mse = mean_squared_error(y_test, xgb_default_pred)
```

The default XGBoost test MSE was `0.0215`.

Interpretation

Default XGBoost is competitive, but it did not beat the tuned random forest or the ridge baseline here.

(b) Learning Rate Versus Number of Trees

I compared several learning rates and tracked train and test MSE as the number of boosting rounds increased.

```
learning_rates = [0.05, 0.3, 0.9]
n_estimators = 150

plt.figure(figsize=(12, 7))

for lr in learning_rates:
    booster = xgb.XGBRegressor(
        n_estimators=n_estimators,
        learning_rate=lr,
        random_state=2026
    )

    booster.fit(
        X_train_transformed,
        y_train,
        eval_set=[
            (X_train_transformed, y_train),
            (X_test_transformed, y_test),
        ],
        verbose=False,
    )

    results = booster.evals_result()
    train_mse = np.square(results["validation_0"]["rmse"])
    test_mse = np.square(results["validation_1"]["rmse"])
```

```
    plt.plot(test_mse, label=f"Test (lr={lr})")
    plt.plot(train_mse, linestyle="--", alpha=0.35, label=f"Train (lr={lr})")

plt.legend()
plt.title("XGBoost MSE by Boosting Iteration")
plt.xlabel("Number of Boosting Iterations")
plt.ylabel("MSE")
plt.show()
```

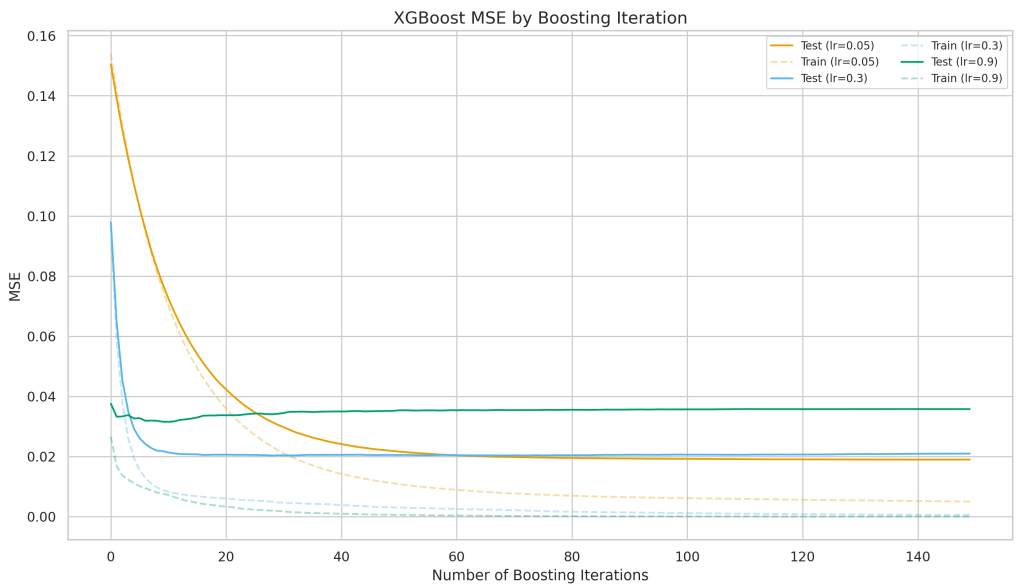

Figure 8: XGBoost training and test MSE over boosting iterations for several learning rates.

The smaller learning rate needed more trees but generalized better, while the highest learning rate improved quickly and then overfit.

Interpretation

The main tradeoff is between speed and stability. Larger learning rates learn quickly but can overfit, while smaller learning rates usually need more trees and generalize better.

Extra Credit: Early Stopping

For the extra-credit analysis, I also compared learning rates under an early-stopping workflow.

```
learning_rates = [0.01, 0.03, 0.1, 0.3, 0.5]

# Example summary from the early-stopping tuning run:
# best learning rate: 0.03
# lowest validation MSE: about 0.0243
```

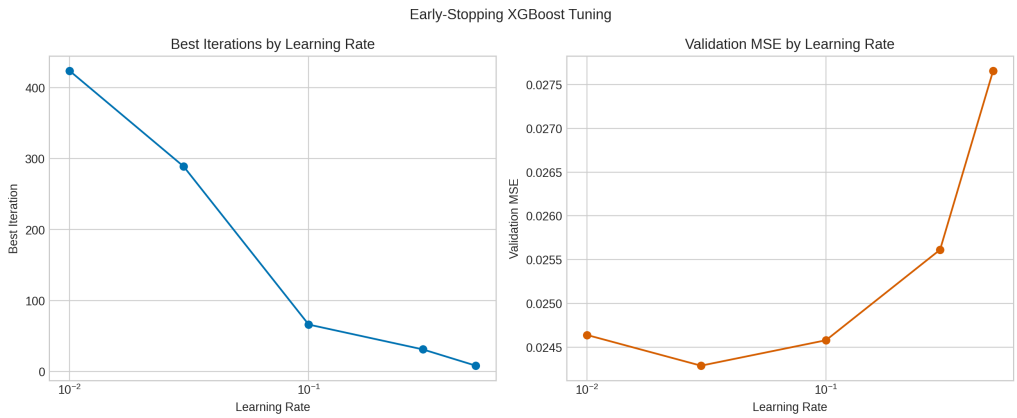

Figure 9: Early-stopping XGBoost tuning curves.

The validation error was lowest around the lower-to-middle learning-rate range rather than at the highest rates.

Interpretation

Even with early stopping, boosting did not outperform the best earlier models in this assignment. The random forest and ridge regression still gave the strongest overall results.

Overall Model Comparison

Model	Test MSE
Ridge regression	0.0198
Simple decision tree	0.0374
Pruned tree	0.0327
Random forest	0.0197
Default XGBoost	0.0215

The best overall model in this lab was the random forest, with ridge regression close behind. Both models were more accurate than the single-tree models and the untuned boosting model, and both consistently identified housing quality and living area as the strongest predictors.